

THE UNIVERSITY OF ZAMBIA
SCHOOL OF NATURAL AND APPLIED SCIENCES
DEPARTMENT OF COMPUTING AND INFORMATICS

FINAL YEAR PROJECT REPORT

ACADEMIC YEAR 2026

**SVDS: A Camera-Based Stolen Vehicle Detection System
for the Zambia Police Service — The Web Portals,
Analytics and Quality Assurance**

A Final Year Project Report Submitted to the
Department of Computing and Informatics
in partial fulfilment of the requirements for the award of

[DEGREE PROGRAMME — confirm]

Student Name: [STUDENT NAME — Role 4: Frontend]

Computer Number: [COMPUTER NUMBER]

Supervisor: [SUPERVISOR NAME]

Date of Submission: [DATE OF SUBMISSION]

DECLARATION

I declare that this final year project report is my own work and has been prepared in accordance with the academic requirements of the University of Zambia. All sources of information, ideas, data, software, images, and other material that are not my own have been appropriately acknowledged and referenced.

Where this project was undertaken as part of a group, I have clearly identified my individual contributions and have not presented the work of other group members as my own.

Student Name: [STUDENT NAME — Role 4: Frontend]

Computer Number: _____

Signature: _____

Date: _____

APPROVAL

This final year project report has been submitted for examination with the approval of the project supervisor.

Supervisor Name: _____

Signature: _____

Date: _____

Departmental/Examiner	Approval	(if	applicable):

ABSTRACT

Stolen-vehicle recovery in Zambia depends largely on police officers chancing to recognise a plate number from a paper report or radio bulletin. This project, carried out by a group of four students, built SVDS, a Stolen Vehicle Detection System on top of the VRRS vehicle registry. The public files stolen-vehicle reports through a web application, and a camera node reads plates from a phone's video stream and checks them against active reports, so connected police officers are alerted in real time.

This individual report covers the front end, analytics, testing and documentation. The front end is a React single-page application with three role-based portals (reportee, police, admin) in fourteen pages, built with four runtime packages. Seven analytics endpoints compute recovery rate, recovery time, detections and alert statistics from stored records, and a shared expression corrects a defect in which detection confidence stored on two scales (fraction and percentage) produced an average of about 30 instead of about 87. The system is supported by 39 automated tests, a README and a run script.

The application builds into a 263.6 kB JavaScript bundle (82.5 kB gzipped), and all 39 Python tests pass. A line-tracing measurement, made with a standard-library tracer because the usual tool could not be installed, showed that the tests run about 63.6% of the backend's lines, with user management (27%) and system routes (32%) least covered. A scripted browser session found that a wrong password gives no error message because the page reloads, and further checks found unhandled loading failures, non-proportional and gappy charts, form labels not associated with inputs, no front-end tests, and README instructions that no longer match the code, including a setting the backend ignores. None of these findings has been fixed in this project. No user evaluation was carried out.

Keywords: React; single-page application; analytics; software testing; code coverage

Note on the version of the code evaluated. All findings, test counts, coverage figures and line counts in this report were obtained against the code as committed (commit a349fc6). After that evaluation, uncommitted changes appeared in the working tree (middleware.py, alerts.py, system.py, main.py, api.js, SystemHealth.jsx, conftest.py, a new tests/test_regressions.py and a new front-end test). With those changes the backend suite has **38 tests, all passing**, and the front end has **one** test (npm test), which also passes. The changes appear to address the items listed below; this was established by reading the changes and running the tests, and the original probes were **not** re-run against the changed code. Where this report says a finding was "not fixed", it refers to the evaluated commit.

Appear addressed in the working tree: **D-1** (the backend now reads CORS_ORIGINS, so the README instruction takes effect), **U-11** in part (one front-end test now exists, for the alert socket helper), and the health page's connection card. *Still open as far as the changes show:* U-1 and U-2 (the 401 handler is unchanged, so the failed-login problem remains), U-3 to U-10, D-2 to D-4, and the low coverage of user management and system routes (coverage was not re-measured).

ACKNOWLEDGEMENTS

[Write acknowledgements here: supervisor, group members (AI/vision, backend and security, real-time and integration), family, and anyone who helped collect plate images. The author must write this; it is personal.]

CONTENTS

Preliminary pages: Declaration · Approval · Abstract · Acknowledgements · List of Abbreviations and Acronyms · Note on Group Projects

1 Introduction — 1.1 Background and Context · 1.2 Problem Statement · 1.3 Project Aim · 1.4 Project Objectives · 1.5 Research Questions · 1.6 Scope of the Project (In Scope, Out of Scope) · 1.7 Significance · 1.8 Limitations · 1.9 Organisation of the Report

2 Literature Review — 2.1 Introduction · 2.2 Key Concepts and Theories · 2.3 Existing Systems / Related Work · 2.4 Comparative Analysis · 2.5 Research / Knowledge Gap · 2.6 Conceptual Framework · 2.7 Chapter Summary

3 Methodology — 3.1 Introduction · 3.2 Research / Development Approach · 3.3 Requirements / Data Collection · 3.4 System Architecture / Research Workflow · 3.5 Tools and Technologies · 3.6 Ethical and Legal Considerations · 3.7 Evaluation Method · 3.8 Chapter Summary

4 System Analysis and Design — 4.1 Introduction · 4.2 Stakeholder and User Analysis · 4.3 Functional Requirements · 4.4 Non-Functional Requirements · 4.5 Use Cases / User Stories · 4.6 System / Solution Architecture · 4.7 Database / Data Design · 4.8 Interface / Interaction Design · 4.9 Algorithms / Models / Technical Design · 4.10 Chapter Summary

5 Implementation / Development — 5.1 Introduction · 5.2 Development Environment · 5.3 System Components / Modules · 5.4 Key Implementation Details · 5.5 Integration · 5.6 Challenges and Solutions · 5.7 Individual Contribution · 5.8 Chapter Summary

6 Testing, Results and Evaluation — 6.1 Introduction · 6.2 Testing Strategy · 6.3 Test Cases and Results · 6.4 Experimental / Performance Results · 6.5 User Evaluation · 6.6 Analysis of Results · 6.7 Objective-by-Objective Evaluation · 6.8 Individual Results · 6.9 Chapter Summary

7 Discussion — 7.1 Introduction · 7.2 Interpretation of Findings · 7.3 Comparison with Existing Work · 7.4 Contributions of the Project · 7.5 Limitations and Implications · 7.6 Chapter Summary

8 Conclusion and Recommendations — 8.1 Introduction · 8.2 Summary of the Project · 8.3 Conclusions · 8.4 Achievement of Objectives · 8.5 Recommendations · 8.6 Future Work

References

Appendices — A Individual Contribution Statement · B User Manual / Installation Guide · C Additional System Designs / Diagrams · D Additional Test Cases and Results · E Data Collection Instruments · F Additional Code / Configuration · G Other Supporting Material

[Page numbers are not shown here. In Word, apply heading styles and use References → Table of Contents to generate a numbered contents page, and a List of Figures and List of Tables the same way.]

LIST OF ABBREVIATIONS AND ACRONYMS

Abbreviation	Meaning
ALPR	Automatic Licence Plate Recognition
API	Application Programming Interface
CUDA	Compute Unified Device Architecture (NVIDIA GPU computing platform)
SPA	Single-Page Application
WCAG	Web Content Accessibility Guidelines
DBMS	Database Management System
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
IoU	Intersection over Union
JWT	JSON Web Token
mAP	mean Average Precision
MJPEG	Motion JPEG (video stream format)
ML	Machine Learning
OCR	Optical Character Recognition
SVDS	Stolen Vehicle Detection System
UI	User Interface
VRRS	Vehicle Registry and Reporting System
YOLO	You Only Look Once (object detector family)

NOTE ON GROUP PROJECTS

Students who worked on a project as a group are strongly encouraged to prepare their own individual final reports rather than submitting identical reports. The project, system, dataset, or overall research problem may be shared, but each student's report should clearly demonstrate that student's own understanding, responsibilities, work, results, and contribution to the project.

In particular, group-project students should:

- state the overall group project and explain how the individual work fits within it;
- clearly identify their own responsibilities and deliverables;
- focus detailed analysis, design, implementation, experiments, testing, and/or evaluation on their own contribution;
- acknowledge the work completed by other group members where it is discussed;
- include an *Individual Contribution Statement* in the appendices; and
- not claim another group member's work, results, code, data collection, or design decisions as their own.

Chapter 1: Introduction

1.1 Background and Context

Vehicle theft is a persistent problem for law enforcement in Zambia. Recovering a stolen vehicle depends heavily on chance: an officer happens to recognise a plate number from a paper report or a radio bulletin. Reports are kept on paper or in disconnected spreadsheets, so a report filed at one station is invisible to an officer elsewhere.

This report describes **SVDS**, the Stolen Vehicle Detection System, built by a group of four students on top of the **VRRS** registry. The public files stolen-vehicle reports, a camera node reads plates from a phone's video stream and checks them against active reports, and police officers are alerted.

None of this is usable unless people can reach it. Members of the public need a simple way to file and follow a report; officers need to search reports, activate them, mark vehicles recovered and act on alerts; administrators need to manage accounts and the system. Someone also has to show whether the system is *working and effective*, with statistics that are true, and to check that the parts of the system do what they claim. That is the work of the **frontend, analytics, testing and documentation** component, and this is the individual report of the student responsible for it.

1.2 Problem Statement

A registry that only its developers can operate does not solve the problem. The specific problems are:

1. **Access.** Three different groups of people (public, police, administrators) need three different, role-appropriate ways into the same data, on an ordinary browser.
2. **Insight.** Police and administrators need statistics (reports by status, recovery rate, recovery time, detections, alert confidence) that are computed from real records and are not misleading.
3. **Assurance.** The group needs evidence that the system behaves as intended, and a way to repeat that check as the code changes.
4. **Usability of the project itself.** Others must be able to install, run and understand the system.

1.3 Project Aim

Group aim. To design, implement and evaluate SVDS: a vehicle registry and reporting web application integrated with a real-time, camera-based licence plate detection pipeline, for the Zambia Police Service and the public it serves.

Individual aim (Frontend, analytics, testing and documentation). To build the three role-based web portals and the analytics that feed them, to verify the system with automated tests, and to document how to run it, and to evaluate these honestly.

1.4 Project Objectives

Group objectives

1. Role-based accounts (Reportee, Police, Admin) with scoped access.
2. Public filing and tracking of stolen-vehicle reports.
3. A live view of alerts, the registry and analytics for police.
4. Administrator control of accounts, system health and the audit trail.
5. Detection and reading of plates from a live camera feed, matched against active reports.
6. Real-time alerts to connected police clients.

Individual objectives (Frontend, analytics, testing and documentation)

No.	Objective	Measure of success
U1	Build a single-page web application with separate Reportee, Police and Admin portals and route guards that mirror the server's roles	Fourteen pages; each role sees only its own navigation
U2	Let the public register, log in, file a report, view their reports and edit their profile	The flows work against the real API
U3	Let police search and filter the registry, activate reports, mark vehicles found, and review and clear alerts	The flows work against the real API
U4	Let administrators manage users and view the audit log and system health	The flows work against the real API
U5	Provide analytics endpoints and a page whose numbers are computed from real records, including correct handling of mixed confidence scales	Values match the data in tests
U6	Verify the system with an automated test suite and measure how much of the backend it exercises	Suite passes; coverage measured
U7	Document how to install and run the system, and provide a run script	Written guide and one-command start
U8	Evaluate the interface and analytics honestly and record defects	Build verified; a browser check; findings listed

1.5 Research Questions

1. Can a single-page application with client-side route guards give each role only the interface it needs, while leaving the real enforcement to the server?
2. Are the analytics figures shown to users correct and unambiguous, and where are they not?

1.6 Scope of the Project

1.6.1 In Scope

- **This report:** the React front end (14 pages, shared layouts, contexts, API client), the analytics endpoints and the page that shows them, the confidence-scale normalisation fix, the backend and node automated tests as a body of evidence, and the run script and documentation.
- **Group:** the whole system.

1.6.2 Out of Scope

- The detection model and camera node (AI/vision member).
- Authentication, roles, the data model and report rules on the server (backend member).
- The WebSocket alert channel, camera status, feed proxy and health endpoint (real-time member). The pages consume them, but their design is covered in that member's report.
- A native mobile app, accessibility certification and internationalisation.

1.7 Significance of the Project

The interface is how the Zambia Police Service and the public meet the system. A clear reporting flow makes reports more complete, and an honest analytics page tells officers whether the system is actually recovering vehicles. A test suite and written instructions let the system be maintained after the students leave.

1.8 Limitations

- **No automated front-end tests.** The interface was checked by building it, reading the code and one scripted browser session; there is no unit, component or browser test suite.
- **Limited browser evidence.** One scripted headless-browser check (the login failure in §6.4) was run for this report. Fifteen screenshots of the running application were captured earlier. No cross-browser, mobile or screen-reader testing was done.
- **Coverage is approximate.** Line coverage was measured with a small tracer built on Python's standard library because the usual coverage tool could not be installed offline; figures may differ slightly from the standard tool.
- **No user evaluation.** No member of the public or officer used the interface for this report.
- **Test authorship.** The tests exercise other members' code; who wrote each file should be confirmed by the student (Appendix A).

1.9 Organisation of the Report

Chapter 2 reviews single-page applications, client-side access control, analytics and testing concepts. Chapter 3 describes the method and tools. Chapter 4 gives the analysis and design. Chapter 5 describes the implementation. Chapter 6 presents testing, browser findings and analytics checks. Chapter 7 discusses them, Chapter 8 concludes, and the appendices contain the contribution statement, user manual, extra designs, test evidence and code.

Chapter 2: Literature Review

2.1 Introduction

This chapter reviews the concepts behind the front end, analytics and testing work: single-page applications, client-side routing and access control, token storage, presenting statistics honestly, and software testing. It then reviews related tools and identifies the gap.

2.2 Key Concepts and Theories

2.2.1 Single-page applications and React

A **single-page application (SPA)** loads once and then updates the page in the browser, calling a server API for data. **React** builds interfaces from components, small functions that return what the screen should show for the current state. **React Router** maps URLs to components in the browser. **Vite** is a build tool and development server; vite build produces static files that can be served by any web server.

2.2.2 State and context

State is data that changes what is shown. React's **context** shares state (for example, the logged-in user) with many components without passing it through each one. SVDS uses two contexts: one for the signed-in user and one for alerts.

2.2.3 Client-side access control

A route guard in the browser hides pages a role should not see. It improves usability but is **not security**: a determined user can call the API directly, and the browser's code is visible. Real enforcement must be on the server. Guards should mirror the server's rules so users are not shown controls that will fail.

2.2.4 Token storage in the browser

A token can be kept in `localStorage` (simple, survives reloads, readable by any script on the page and so exposed to cross-site scripting), in an HTTP-only cookie (not readable by scripts, but needs protection against cross-site request forgery), or in memory (safest, lost on reload). React escapes text by default, which reduces cross-site scripting risk.

2.2.5 Presenting statistics

Analytics are only useful if they are correct and unambiguous. Common pitfalls are: mixing units (a fraction and a percentage), dividing by the wrong base (a "recovery rate" whose denominator includes

cases not yet worked), charts whose axes do not start at zero or that hide empty periods, and counting things by searching text instead of recording them.

2.2.6 Testing concepts

Unit tests check small pieces in isolation; **integration tests** check parts working together (here, the real API routes with a test database); **regression tests** keep a fixed bug from returning; **end-to-end tests** drive the whole system through the interface. **Code coverage** measures which lines a test suite runs. It shows what is *untested*, not what is *correct*: a line can run without its result being checked.

2.2.7 Accessibility

The Web Content Accessibility Guidelines (WCAG) ask, among other things, that form fields have programmatically associated labels so assistive technology can announce them, and that errors are visible and identified in text.

2.3 Existing Systems / Related Work

- **Component libraries and admin frameworks** (Material UI, Ant Design, React-Admin) give ready-made tables, forms and charts. This project hand-writes its layout and charts with plain CSS, which keeps dependencies to four runtime packages (React, React DOM, React Router, Axios) at the cost of building and testing everything itself.
- **Charting libraries** (Chart.js, Recharts) handle scales, axes and empty periods. This project draws bars and a donut with CSS.
- **Testing tools:** pytest for Python; Jest and React Testing Library for components; Playwright or Cypress for browser tests. Only pytest is used here.
- **Coverage tools:** coverage.py is the standard for Python.
- **Existing police records interfaces** are mostly closed, so no comparison of interface quality is possible.

Note on sources. This chapter relies on public documentation and standards listed in the References. No usability figures from other systems are quoted.

2.4 Comparative Analysis

Criterion	This project	With a component and chart library
Runtime dependencies	4	Many more
Bundle size (measured)	263.6 kB JS, 13.5 kB CSS (82.5 kB JS gzipped)	Typically larger
Chart correctness handled for you	No (hand-written)	Mostly yes

Criterion	This project	With a component and chart library
Accessibility built in	No	Partly
Effort to build	Higher per feature	Lower
Automated front-end tests	None	Usually added

The bundle figures are from the production build run for this report; the right-hand column is a general expectation, not a measurement.

2.5 Research / Knowledge Gap

Student projects often show screenshots and stop. The gap this report addresses is an **evaluation of the interface and analytics against their own claims**: what the numbers mean, where they can mislead, and what a real browser does when things go wrong, together with a measured statement of what the test suite does and does not cover.

2.6 Conceptual Framework

```

browser (React SPA) --Axios + Bearer token--> API --> analytics queries --> numbers on
screen
      |
      |
route guards (mirror the server's roles)      ^
contexts: user, alerts                        |
                                             tests check the numbers and the rules

```

The interface reflects the server's rules; the analytics turn records into figures; the tests check both.

2.7 Chapter Summary

The front end is a small hand-built React SPA. Route guards and hand-written charts are simple and keep dependencies low, but shift responsibility for correctness, accessibility and testing onto the team. The rest of the report evaluates how well that responsibility was met.

Chapter 3: Methodology

3.1 Introduction

This chapter explains how the front end, analytics, tests and documentation were developed and evaluated: the approach, requirements, architecture, tools, ethics and evaluation method.

3.2 Research / Development Approach

The project was built **iteratively**. The front end was developed against the running API: each page was built, tried in the browser against the real backend, and adjusted. Analytics endpoints were written alongside the page that shows them, so each chart had real data behind it from the start; the project's working rule was that a statistic on screen must be computed from stored records, not typed in.

Testing followed a **find, fix, lock-in** pattern for defects: a suspected defect was checked against the running system, fixed, and covered by a regression test (for example the confidence-scale fix, §5.4.2). For this report a further **verify-what-was-claimed** step was added: the production build was run, coverage was measured, suspected interface and analytics defects were checked in a real browser or with probe tests, and only observed behaviour was recorded.

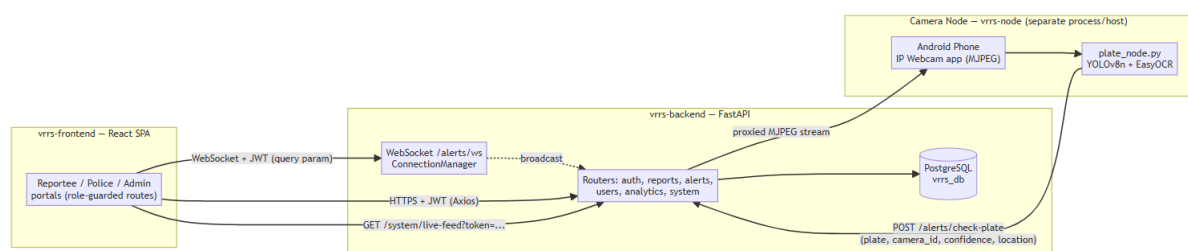
Requirements → Design → Implementation → Integration → Testing / Evaluation → (issues found? back)

3.3 Requirements / Data Collection

Requirements came from (a) the gap in Chapter 1, (b) the three groups of users and what each needs to do, and (c) what the server offers. No interviews, surveys or usability sessions are recorded, so the requirements are the group's own analysis.

The analytics use the system's own records: reports, alerts, users and the audit log. Test data is synthetic. Screenshots of the running system (Chapter 4) were captured earlier from the live application with demonstration accounts.

3.4 System Architecture / Research Workflow



The front end is one of three independent processes; it talks to the backend by HTTPS/JSON with a Bearer token and by WebSocket for alerts.

3.5 Tools and Technologies

Area	Technology
Front end	React 18.2, React Router 6.22, Axios 1.6 (the only runtime dependencies)
Build and dev server	Vite 5 with <code>@vitejs/plugin-react</code> ; dev server on port 5173
Styling	One hand-written stylesheet (<code>styles.css</code> , 1,084 lines) plus inline styles on the login and register pages
Charts	Hand-made with CSS (bars, conic-gradient donut); no charting library
Analytics queries	SQLAlchemy aggregate queries in FastAPI routes
Testing	pytest with FastAPI <code>TestClient</code> ; Python's standard-library tracing for approximate line coverage
Browser check	Headless Chrome driven over the DevTools Protocol from a short Python script
Documentation and start-up	<code>README.md</code> , <code>run-all.ps1</code>
OS	Windows 11

3.6 Ethical and Legal Considerations

- **Personal data on screen.** Pages show names, telephone numbers, plates and locations. Server-side role checks decide who receives them; the interface's guards only avoid showing controls that would fail.
- **Token storage.** The token is kept in the browser's `localStorage` (readable by any script running on the page). React escapes displayed text and the code does not use `dangerouslySetInnerHTML`, which lowers, but does not remove, the risk.
- **Analytics honesty.** Statistics can mislead officers and the public if they are wrongly defined or drawn. §6.4 records where they can.
- **Accessibility.** Fields without associated labels are hard for screen-reader users; this was checked on the login page (§6.4).
- **Responsible testing.** The browser check used a made-up email address and password, which the server rejected; nothing was written to the database. Probes ran on an in-memory database. Servers started for the check were stopped afterwards.
- **Test data.** Tests use synthetic names and plates.

3.7 Evaluation Method

1. **Build check:** run the production build and record its output.

2. **Automated tests:** run the 31-test backend suite and 8-test node suite; measure line coverage of the backend package.
3. **Browser check:** start the backend and dev server, drive headless Chrome through a failed login, and record what appears and what network calls are made.
4. **Analytics probes:** temporary tests with edge-case data to see what each analytics endpoint returns.
5. **Code review** of the pages for error handling, keys, labels and search behaviour.
6. **Objective-by-objective review** against U1 to U8 in §1.4.

Not done: cross-browser, mobile, load, screen-reader or user evaluation.

3.8 Chapter Summary

The interface and analytics were built against the live API and checked by running the build, the tests, a scripted browser session and edge-case probes. What was and was not observed is stated in Chapter 6.

Chapter 4: System Analysis and Design

4.1 Introduction

Stakeholders and the architecture are shared by the group. The requirements, interface design and analytics design below are those of the **frontend, analytics, testing and documentation** component.

4.2 Stakeholder and User Analysis

Stakeholder	Goal	What the interface must give
Reportee (public)	Report a theft and follow it	Simple registration and login; a report form; a list of their own reports with status; profile editing
Police officer	Recover vehicles	Registry search and filter; activate and mark found; live alerts to review or dismiss; camera view; analytics
Administrator	Run the system	User list with role and account controls; audit log; system health
Maintainer	Keep the system working	Tests, documentation, one-command start

4.3 Functional Requirements

ID	Requirement
FR-U1	Login and registration pages; after login, redirect to the portal for the user's role.
FR-U2	Route guards: reportee pages require any signed-in user; police pages require police or admin; admin pages require admin; otherwise redirect to the user's home.
FR-U3	Reportee portal: home, report-vehicle form, my reports (with status), profile (edit details, change password).
FR-U4	Police portal: vehicle registry with search and status filter and activate / mark-found actions; live alerts with mark-read, mark-all-read and false-positive; cameras; analytics.
FR-U5	Admin portal: overview, user management (invite police, change role, deactivate, delete), system health, audit log.
FR-U6	Attach the token to every API request; on an authentication failure clear the session and go to the login page.
FR-U7	Analytics endpoints for summary counts, reports per month, status breakdown, detections per camera, alerts over time, recovery time and user growth.
FR-U8	Every figure shown by analytics is computed from stored records. Confidence scores stored on different scales are normalised before averaging.
FR-U9	The repository has instructions for installing and running each part, and a script that starts all three.

4.4 Non-Functional Requirements

- **Consistency with the server:** the interface's role rules mirror the server's, which remains the authority.
- **Simplicity and low dependency count:** four runtime packages.
- **Clarity:** errors from the server are shown to the user in plain text where the code handles them.
- **Correctness of statistics:** unambiguous definitions; mixed units handled.
- **Repeatability:** tests can be run with one command and do not touch the real database.
- **Maintainability:** shared layouts (PortalLayout, ReporteeLayout), one API module, two contexts.

4.5 Use Cases / User Stories

UC-U1: A member of the public reports a theft. They register, log in, fill in the report form (plate, vehicle details, incident date, last seen, description), submit, and later see the report in "My reports" with its status.

UC-U2: An officer works a report. They search the registry by plate or owner, filter by status, activate an under-review report so cameras can match it, and mark it found when the vehicle is recovered.

UC-U3: An officer reviews alerts. They open Live alerts, see unread items, and mark them read, mark all read, or flag a false positive.

UC-U4: An officer reviews performance. On Analytics they see totals, average recovery time, average detection confidence, false positives, reports per month, the status breakdown and detections by location.

UC-U5: An administrator manages users. They invite a police officer, change a role, deactivate an account or delete one, and read the audit log.

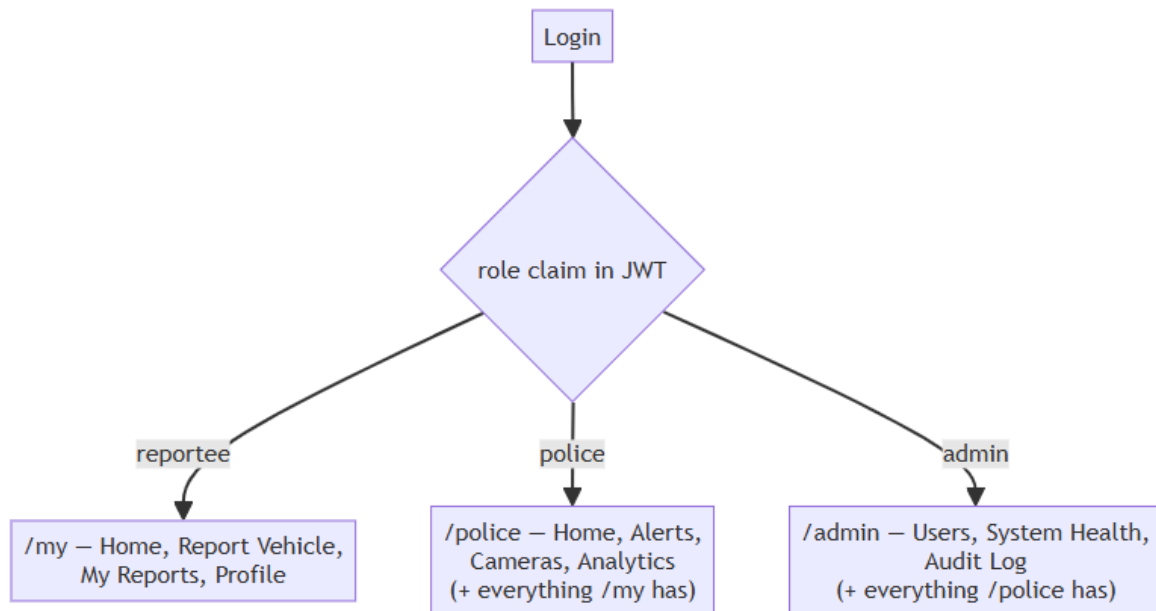
User story. *As a police commander, I want the recovery rate and average confidence on the analytics page to be computed from real records, so that I can trust them when reporting to my superiors.*

4.6 System / Solution Architecture

src/	
main.jsx, App.jsx	entry point, providers, routes with role guards
context/AuthContext.jsx	signed-in user, login/logout, session restore
context/AlertsContext.jsx	alert connection and unread count (real-time member)
services/api.js	Axios instance, token header, 401 handler, API groups
components/	ProtectedRoute, PortalLayout, ReporteeLayout,
NotificationToast	

config/policeNav.js pages/	police navigation with unread badge Login, Register, reportee/, police/, admin/ (14 pages)
-------------------------------	---

App.jsx wraps everything in AuthProvider and AlertsProvider, mounts the toast once, and declares fourteen routes, each inside a Guard with the roles allowed.



4.7 Database / Data Design

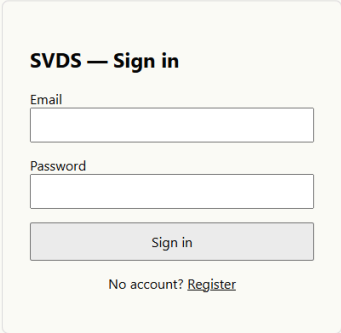
The front end has no database. The analytics read the backend's four tables (users, reports, alerts, audit log), whose design is in the backend member's report (Appendix C). The analytics **derived values** are defined here:

Figure	Definition in the code
Recovery rate	$\text{found} / \text{total reports} \times 100$, where total includes under_review and missing reports
Average recovery time	Mean of $(\text{resolved_date} - \text{report_date})$ in whole days over reports that are found and have a resolved date
Average detection confidence	Mean of normalised confidence over all alerts, including ones later flagged as false positives
False positives	Number of audit-log rows whose text contains "false positive"
Reports per month	Count of reports by calendar month for the last $n \times 30$ days; months with no reports are omitted
Alerts over time	Count of alerts per day for the last n days; days with none are omitted
Detections per node	Alert counts grouped by camera id and location text

Confidence normalisation. Older detections stored confidence as a fraction (0 to 1) and the current camera node stores a percentage (0 to 100). Every average first converts any value at or below 1 to a percentage ($\text{value} \times 100$).

4.8 Interface / Interaction Design

The pages were captured from the running application. They are reproduced here as evidence of the interface.



The image shows a sign-in form titled "SVDS — Sign in" centered on a light yellow background. The form contains two input fields: "Email" and "Password". Below these fields is a "Sign in" button. At the bottom of the form, there is a link that says "No account? [Register](#)".

SVDS — Sign in

Email

Password

No account? [Register](#)

SVDS — Create account

first name *

last name *

email *

phone *

+260 97 1234567

Account type

Public Reportee

Police and admin accounts are created by administrators.

National ID *

123456/78/1

Password

Confirm password

Create account

Have an account? [Sign in](#)

SVDS

My Account

Home

Report vehicle

My reports

My profile

DR

Demo Reportee

Public user

Logout

Welcome back, Demo

Track your vehicle reports and their current status

Total reports filed

1

Since Jan 2026

Currently missing

0

Active cases

Recovered

0

Vehicle returned

Recent reports

+ New report

PLATE

VEHICLE

REPORTED

STATUS

**DEM
0001**

Toyota
Corolla -
Silver

2026-09-
16

under
review

View

SVDS

My Account

Home

Report vehicle

My reports

My profile

DR

Demo Reportee

Public user

Logout

Report vehicle

Tell us what was stolen so police can verify and activate the case.

Vehicle details

License plate *

e.g. BAA 1234

Color

e.g. White

Make *

e.g. Toyota

Model *

e.g. Land Cruiser

Year

e.g. 2019

Incident details

Date stolen

dd/mm/yyyy

Last seen location *

e.g. Lusaka CBD, Cairo Roac

SVDS

My Account

Home

Report vehicle

My reports

My profile

DR

Demo Reportee

Public user

Logout

My reports

Review every vehicle report connected to your account.

+ New report

DEM 0001 - Toyota Corolla

under review

Filed: 2026-09-16 - Last seen: Lusaka, Manda Hill

This is demo report — parked overnight, gone by morning.

SVDS

Police Portal

Vehicle registry

Live alerts

Cameras

Analytics

VR

Zambia Police

Police Portal

Logout

Vehicle registry

Manage stolen and recovered vehicle reports

Total reports

18

+3 this week

Missing

3

Open cases

Recovered

13

All time

Recovery rate

72.2%

All time

All reports

Search plate or c

All status

+ Add vehicle

PLATE	VEHICLE	REPORTED	STATUS	ACTIONS
DEM 0001	Toyota Corolla - Silver	2026-09-16	under review	Activate

toyota

SVDS

Police Portal

Vehicle registry

Live alerts

Cameras

Analytics

VR

Zambia Police

Police Portal

Logout

Live alerts

Real-time camera detections - updates via WebSocket

Unread alerts

0

Live

Alerts today

0

Across 0 nodes

Alert feed

Show 25

Unread

History

No unread alerts.

SVDS

Police Portal

Vehicle registry

🔔 Live alerts

📷 Cameras

📊 Analytics

VR **Zambia Police**
Police Portal

Logout

Analytics

Vehicle recovery statistics and detection trends

Total reports

18

+3 this week

Avg. recovery
time

27.7d

13 vehicles
recovered

Avg.
detection
confidence

88.2%

Camera nodes

False
positives

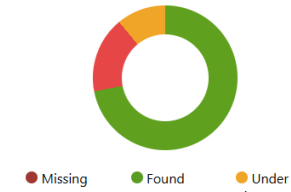
3

Flagged by
officers

Reports per month



Status breakdown



SVDS

Admin Portal

🏠 Overview

👤 User management

🕒 System health

Audit log

VR **Zambia Police**
Admin Portal

Logout

System overview

SVDS platform - Zambia Police Service

Total users

21

+2 this week

Police
accounts

4

4/4 active

Camera
nodes

0/0

0 offline

Server
uptime

0m

Since last
restart

User breakdown

Public
(Reportee)

15

Can file vehicle
reports

Police officers

4

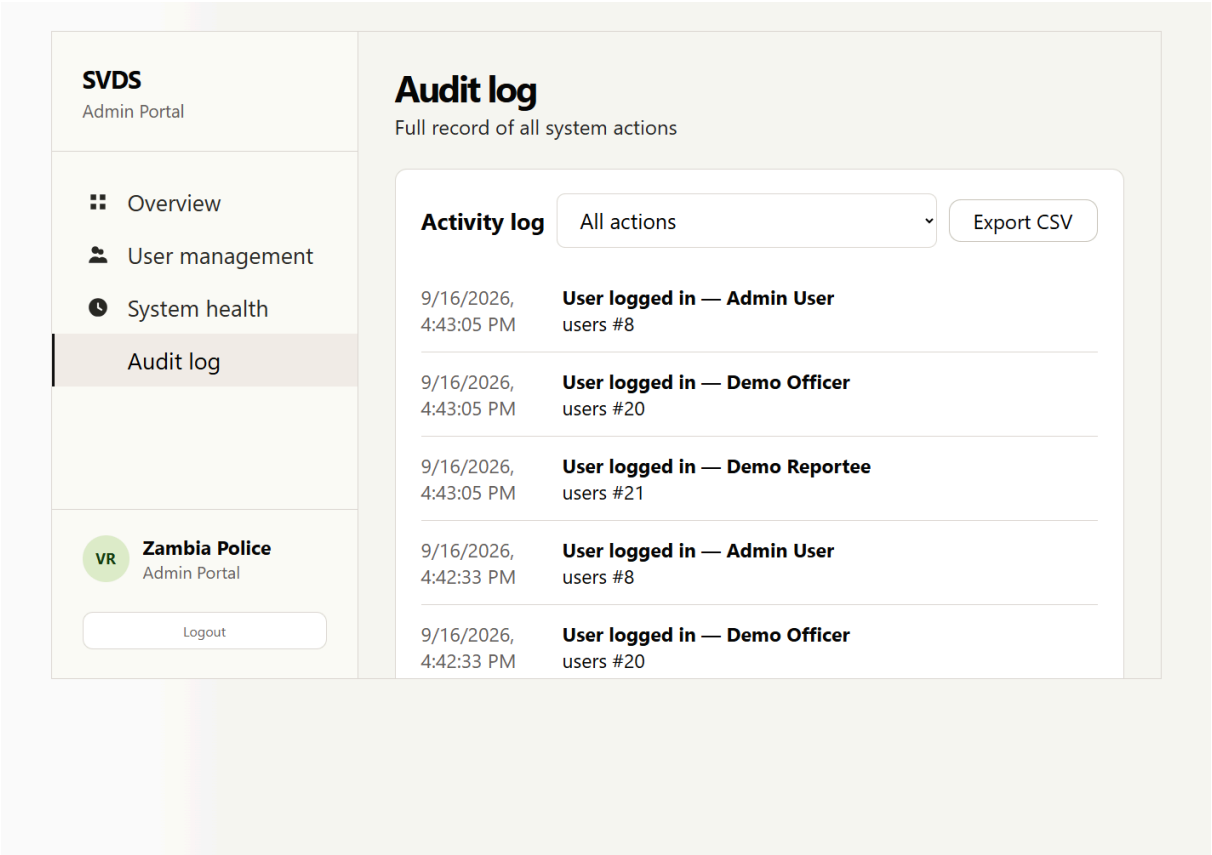
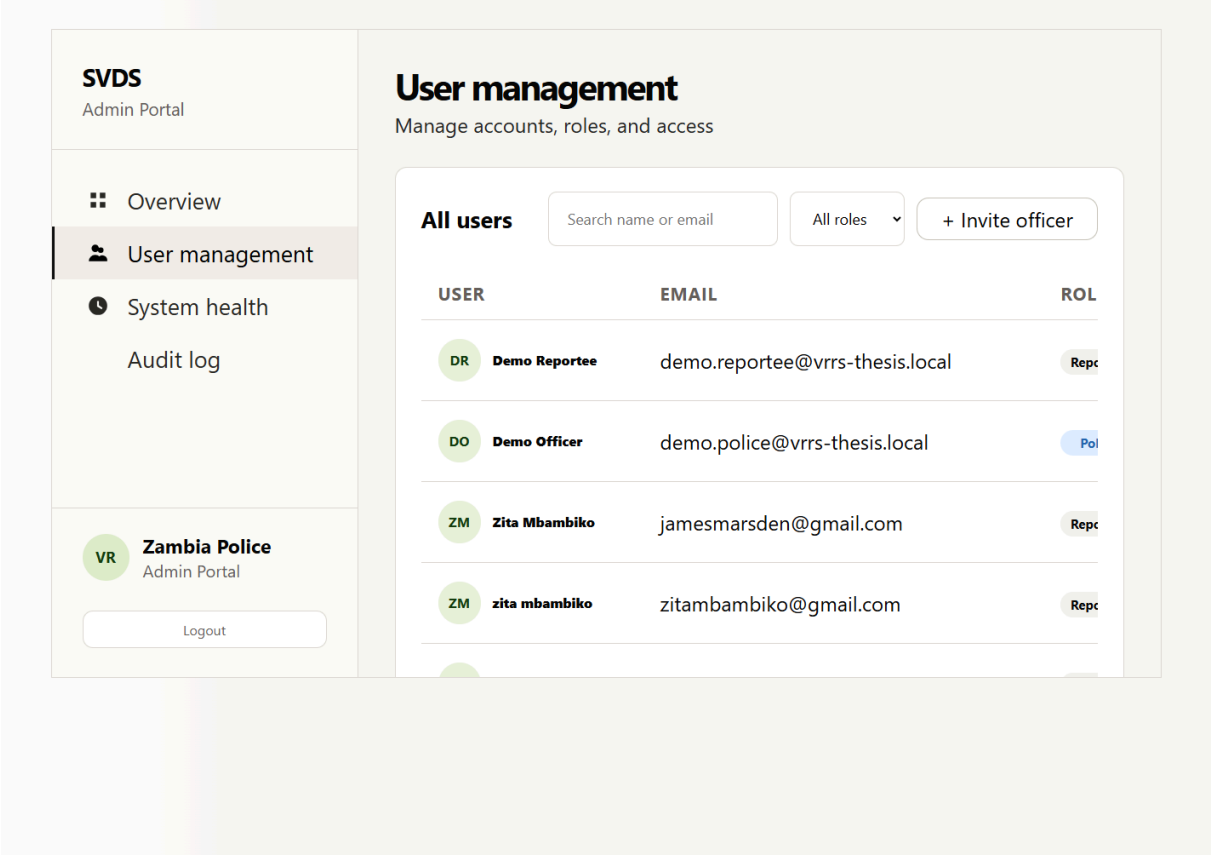
Registry and alert

Quick actions

Manage user accounts ->

View system health ->

Invite new police officer ->



Further screens (profile, cameras, system health, interactive API documentation) are in Appendix C.

Interaction design choices: each role has its own navigation (police navigation shows an unread badge); a toast notification appears on any page; forms show server error text; the register page validates fields in the browser before sending; destructive user deletion asks for confirmation.

4.9 Algorithms / Models / Technical Design

- **Route guard:** `ProtectedRoute` waits for the session to load, redirects to login if no user, and redirects to the user's own home if their role is not allowed.
- **Session restore:** on load, the saved user and token are read from `localStorage`, the user is set at once, and `GET /users/me` refreshes it; failure logs the user out.
- **Request pipeline:** an Axios request interceptor adds the token; a response interceptor treats any HTTP 401 as an expired session (clear storage, go to `/login`).
- **Search:** the registry sends a request on each keystroke with the current search and status.
- **Charts:** bar height = $\max(20, \min(\text{count} \times 10, 170))$ pixels; the donut is a CSS conic gradient from rounded percentages of found and missing (the remainder is drawn as "under review").
- **Analytics queries:** SQL aggregates with `func.count`, `func.avg`, `func.date` and `extract`; the confidence normalisation is one shared expression.

4.10 Chapter Summary

The front end is fourteen guarded pages over one API module and two contexts. Analytics figures have explicit definitions in the code, and a shared expression normalises confidence scales. The evaluation in Chapter 6 tests those definitions against edge cases.

Chapter 5: Implementation / Development

5.1 Introduction

This chapter describes how the front end, analytics, tests and documentation were built, how they connect to the other members' work, the problems met, and the student's contribution.

5.2 Development Environment

Item	Detail
Machine	Windows 11 PC
Front end	Node.js with npm; Vite 5.4 (the production build in §6.4 ran with Vite 5.4.21)
Backend for tests	Python 3.12 virtual environment with pytest
Browser	Google Chrome (development, and headless for the scripted check)
Version control	Git

5.3 System Components / Modules

Module	Size	Purpose	Built by
vrrs-frontend/src (24 JS/JSX files)	1,786 lines	The React application	Frontend (this report)
src/styles.css	1,084 lines	All styling	Frontend (this report)
services/api.js		Axios instance, token header, 401 handler, grouped API calls	Frontend (this report)
context/AuthContext.jsx, components/ProtectedRoute.jsx, App.jsx		Session and guards	Frontend (this report)
context/AlertsContext.jsx, components/NotificationToast.jsx, createAlertSocket		Alert connection and toast	Real-time member
app/routers/analytics.py	87 lines	Seven analytics endpoints	Frontend/analytics (this report)
vrrs-backend/tests/, vrrs-node/tests/	31 + 8 tests	Automated tests	See §5.7
README.md, run-all.ps1	91 lines	Documentation and start-up	Frontend/documentation (this report), with run-all.ps1

Module	Size	Purpose	Built by
			also described in the real-time report

5.4 Key Implementation Details

5.4.1 Routing and guards

```
const Guard = ({ roles, children }) => <ProtectedRoute allowedRoles={roles}>{children}
</ProtectedRoute>;
<Route path="/police/alerts" element={<Guard roles=["police","admin"]><Alerts />
</Guard>} />
<Route path="/admin/users" element={<Guard roles=["admin"]><UserManagement />
</Guard>} />
```

ProtectedRoute shows "Loading..." while the session loads, redirects to /login if nobody is signed in, and redirects to the user's own home if the role is not allowed. Reportee routes allow all three roles, so officers and admins can also file reports.

5.4.2 Analytics and the confidence fix

All averages use one expression:

```
NORMALIZED_CONFIDENCE = case(
  (Alert.confidence_score <= 1, Alert.confidence_score * 100),
  else_=Alert.confidence_score,
)
```

The defect. Detections from a retired standalone script stored confidence as a fraction (0.9), while the camera node stores a percentage (90.1). On the live database, rows for the old camera ids averaged about 0.9 and rows for the current node about 87, so averaging the raw column reported roughly 30 instead of about 87. **The fix** scales any value at or below 1 up to a percentage before averaging, and was chosen over rewriting the old rows so historical data stays untouched. Two regression tests cover it: a mixed 0.8 and 60.0 pair must average 70.0, and a lone 0.5 must display as 50.0. The same expression is used in the camera status code (real-time member).

The other analytics endpoints are direct SQL aggregates (counts by status, by month, by day, by camera; mean recovery time in Python over found reports).

5.4.3 The API module

api.js builds one Axios instance whose base address comes from VITE_API_BASE_URL, adds Authorization: Bearer <token> from localStorage, and groups calls into authAPI, reportsAPI, alertsAPI, usersAPI, analyticsAPI and systemAPI. A response interceptor treats any 401 as an expired

session: it clears `localStorage` and sets `window.location.href = "/login"`. (This behaviour is the cause of finding U-1, §6.4.)

5.4.4 Pages

- **Registry (PoliceHome):** loads reports and the summary, sends a new request on each search or status change, and calls `activate` and `mark-found` then reloads the list.
- **Alerts:** loads unread or all alerts with a page size, prepends live events, and updates the unread count when items are cleared.
- **Analytics:** loads five endpoints together, shows an error box with a `Retry` button if any fails, and draws the bars and donut.
- **Report vehicle:** upper-cases and trims the plate, sends `null` for an empty incident date, and shows the server's error message on failure.
- **Register:** validates fields (including matching passwords) in the browser before sending.
- **User management:** search and role filter; invite, role change, activate/deactivate, delete with a confirmation dialog.

5.4.5 Tests and documentation

The backend tests use FastAPI's `TestClient` with an in-memory SQLite database, an overridden database dependency, and fixtures that create users and tokens for each role (Chapter 6 of the backend report describes the set-up). The node tests cover the pure plate functions. `README.md` gives quick-start steps, the role table, the API list and the camera-node contract; `run-all.ps1` starts all three programs.

5.5 Integration

- **With the backend:** every API group in `api.js` maps to a backend router; the login response supplies the token, role and redirect path.
- **With the real-time member:** the alerts context and toast are mounted by `App.jsx`; the Cameras and System Health pages call the status endpoints.
- **With the AI/vision member:** the node's confidence scale (percentage) is what the analytics normalisation assumes.
- **Configuration:** `VITE_API_BASE_URL` and `VITE_WS_URL` in the front-end `.env`.

5.6 Challenges and Solutions

Challenge	Cause	Solution
Analytics average confidence was wrong (about 30 instead of 87)	Two storage scales in one column	Normalise in SQL before averaging; two regression tests
Charts and numbers had to come from real data	Early versions used fixed sample values	Every figure is computed by an endpoint; empty states ("No detections yet") handled

Challenge	Cause	Solution
Each role needs its own navigation and layout	Three audiences on one app	Shared <code>PortalLayout</code> , <code>ReporteeLayout</code> , and a <code>policeNav</code> config
Browser cannot send a header on a WebSocket or <code></code>	Browser limits	Query-string token (discussed in the real-time report)
Tests must not touch PostgreSQL	<code>database.py</code> connects at import	Dependency override with in-memory SQLite (backend report)
Getting three programs to run together	Separate environments	<code>run-all.ps1</code> and README steps (with the gaps listed in §6.4)

5.7 Individual Contribution (group project)

The student was responsible for **work areas 6 and 7** of the group's seven (Appendix A):

- the React front end: routing and guards, the API module, the three portals and their fourteen pages, shared layouts and the stylesheet;
- the analytics endpoints, the analytics page, and the confidence-scale fix with its regression tests;
- the automated test suites as a body of evidence (running, extending and reporting them), and the coverage measurement in this report;
- the README and the run script;
- the evaluation in Chapter 6.

Work by others that this component uses, **not claimed here**: the token, roles, data model and report rules (backend member); the alert channel, camera status, feed proxy and health endpoint (real-time member); the detection model and camera node (AI/vision member).

Test authorship. The role split assigned "tests" to this role, but the suites also test other members' code. **The student should state which test files they wrote themselves**; this report claims the analytics tests, the running and measuring of the suites, and the coverage analysis.

5.8 Chapter Summary

The front end is a small hand-written React application with fourteen guarded pages. The analytics are SQL aggregates with one shared confidence normalisation, fixed after a real defect. Tests and documentation were assembled around them.

Chapter 6: Testing, Results and Evaluation

6.1 Introduction

This chapter reports what was measured and observed for the front end, analytics, tests and documentation. Every result comes from a command or script run while preparing this report. Temporary probe tests were deleted after running, so the shipped suites are unchanged.

6.1.1 Version of the code evaluated

Note on the version of the code evaluated. All findings, test counts, coverage figures and line counts in this report were obtained against the code as committed (commit a349fc6). After that evaluation, uncommitted changes appeared in the working tree (middleware.py, alerts.py, system.py, main.py, api.js, SystemHealth.jsx, conftest.py, a new tests/test_regressions.py and a new front-end test). With those changes the backend suite has **38 tests, all passing**, and the front end has **one** test (npm test), which also passes. The changes appear to address the items listed below; this was established by reading the changes and running the tests, and the original probes were **not** re-run against the changed code. Where this report says a finding was "not fixed", it refers to the evaluated commit.

Appear addressed in the working tree: **D-1** (the backend now reads CORS_ORIGINS, so the README instruction takes effect), **U-11** in part (one front-end test now exists, for the alert socket helper), and the health page's connection card. *Still open as far as the changes show:* U-1 and U-2 (the 401 handler is unchanged, so the failed-login problem remains), U-3 to U-10, D-2 to D-4, and the low coverage of user management and system routes (coverage was not re-measured).

6.2 Testing Strategy

Level	Technique	Target
Build	vite build	The front end compiles
Automated tests	pytest (31 backend, 8 node)	API behaviour and pure logic
Coverage	Standard-library line tracing across all threads	The backend package
Browser check	Headless Chrome over the DevTools Protocol against the running app	Failed login behaviour, label association
Analytics probes	Temporary tests with edge-case data	The analytics endpoints
Code review	Reading pages and the README	Error handling, keys, search, documentation
User evaluation	None	See §6.5

6.3 Test Cases and Results

Table 6.1: Test cases for this component

ID	Test case	Expected	Observed	Status
TC-U1	Production build	Compiles	107 modules transformed; <code>index.html</code> 0.43 kB, CSS 13.50 kB (3.53 kB gzipped), JS 263.64 kB (82.49 kB gzipped); built in 2.31 s	Pass
TC-U2	Backend suite	All pass	31 passed	Pass
TC-U3	Node suite	All pass	8 passed	Pass
TC-U4	Summary averages mixed confidence scales (0.8 and 60.0)	70.0	70.0 (existing test)	Pass
TC-U5	Detections per node normalises a fraction (0.5)	50.0	50.0 (existing test)	Pass
TC-U6	Reports per month with reports in months 1, 2 and 4 ago	Each month present	Months 1, 2 and 4 present; the empty third month is omitted	Pass with finding U-6
TC-U7	Recovery time for one found report resolved 6 days after filing	6 days	<code>avg_days</code> 6.0, fastest 6, slowest 6, recovered 1	Pass
TC-U8	Analytics query bounds (<code>months=0</code> , <code>days=3</code>)	Rejected	422 for both	Pass
TC-U9	Wrong-password login in a real browser	Error message shown	Not shown: page reloaded (§6.4, U-1)	Fail
TC-U10	Labels on the login form associated with inputs	Associated	0 of 2 labels had a <code>for</code> attribute	Fail (U-3)
TC-U11	Automated front-end tests	Exist	None exist	Not present

6.4 Results and Findings

6.4.1 Coverage of the backend by the automated tests

Measured with a small all-threads line tracer (it counts executable lines run at least once; a run line is not proof its result was checked). The usual tool, `coverage.py`, could not be installed offline, so these figures are approximate.

File	Lines run / executable	%
models.py	61 / 61	100.0
schemas.py	114 / 114	100.0
routers/auth.py	42 / 43	97.7
auth.py	24 / 26	92.3
middleware.py	23 / 27	85.2
main.py	31 / 39	79.5
database.py	12 / 16	75.0
routers/reports.py	65 / 87	74.7
routers/analytics.py	50 / 72	69.4
routers/alerts.py	75 / 110	68.2
routers/system.py	37 / 117	31.6
routers/users.py	31 / 115	27.0
model.py, routers/model.py (unused placeholder)	0 / 61	0.0
Total	565 / 888	63.6

Excluding the two unused placeholder files, coverage is 565 / 827, about 68%. The weakest areas are **user management** (27%: invite, role change, deactivate, delete and profile edits are largely untested) and **system** routes (32%; the real-time member's probes cover part of these but are not part of the suite). There is no coverage figure for the front end because it has no tests.

6.4.2 Interface findings

ID	Finding	Evidence	Severity
U-1	A failed login never shows its error. With a wrong password the server returns 401, the response handler clears storage and reassigns the location to <code>/login</code> , and the page reloads before the login page's error message can be read	Browser check: the login response statuses were <code>[200, 401]</code> (the page load and the failed login), the page's own marker variable was gone after the click (page reloaded), the page text showed only the empty form with no error, and two frame navigations occurred	High (usability)
U-2	Root cause of U-1: every 401 is treated as an expired session , including a failed login	Read from <code>api.js</code>	(same)
U-3	Login form labels are not associated with their inputs	Browser check: 0 of 2 labels have a <code>for</code> attribute. The Register page does associate its labels (read from code)	Medium (accessibility)
U-4	Several data loads have no error handling: Alerts, Cameras, Police home,	Read from code, not run	Medium

ID	Finding	Evidence	Severity
	and System Health call the API without a failure branch, so a failed request leaves "Loading" or an empty page with no message. (Analytics does handle failure with a Retry button)		
U-5	The registry search sends a request on every keystroke with no delay, and nothing guards against replies arriving out of order	Read from code	Low
U-11	No automated front-end tests	No test files exist	Medium

6.4.3 Analytics findings

ID	Finding	Evidence	Severity
U-6	Months and days with no data are omitted , so the "Reports per month" bars skip empty months (June was missing in the probe), which misrepresents the time axis. The bars are also not proportional : height is $\max(20, \min(\text{count} \times 10, 170))$ px, so counts of 1 and 2 look the same and counts of 17 or more look the same	Probe; arithmetic from the code	Medium
U-7	The detections table splits one camera into several rows when its location text varies ("Cairo Rd" and "Cairo Road" gave two rows), and the page uses the camera id as the React key, which would then repeat	Probe; key read from code. A duplicate-key warning was not observed in a browser	Low
U-8	A stored confidence of exactly 1.0 is treated as a fraction and displayed as 100% , though it could mean 1%	Probe: stored 1.0 displayed as 100.0	Low. The rule is a heuristic
U-9	"False positives" is counted by searching audit text for the phrase, so any audit row containing it is counted	Probe: an unrelated audit row containing the phrase raised the count to 1	Low
U-10	Definitions worth stating on screen : the recovery rate divides found reports by <i>all</i> reports including those not yet worked (the probe gave 20% for 1 found of 5), and average confidence includes alerts later flagged as false	Probe and code	Low

6.4.4 Documentation findings

Checked by searching README.md and reading it against the code:

ID	Finding	Severity
D-1	The README tells the reader to set <code>CORS_ORIGINS</code> and, for deployment, to set it to the deployed front end's address. The backend does not read this setting , so following the README would leave a deployed front end blocked (backend report, F-9)	High for deployment
D-2	The README never mentions <code>run-all.ps1</code> , <code>create_admin.py</code> (the only way to create the first administrator), the tests, or the phone camera app	Medium
D-3	The README's camera example shows a <code>NODE-001</code> camera id and a <code>STOLEN/CLEAR</code> response but not <code>alert_id</code> or the <code>vehicle</code> details the server actually returns	Low
D-4	<code>run-all.ps1</code> contains an absolute path to the developer's own Python environment	Medium (portability)

6.4.5 What was not tested

Other browsers, mobile screens, keyboard or screen-reader use, page load time, a live alert on a real screen, behaviour with large data sets, and any user task. The other pages were not driven through the browser in this session; the screenshots in Chapter 4 were captured earlier from the running application.

6.5 User Evaluation

None carried out. No officer, reportee or administrator used the interface for this report, and no survey or interview data exist.

6.6 Analysis of Results

- **The build and test suites are healthy:** the front end compiles into a small bundle (82 kB gzipped JavaScript), and all 39 Python tests pass.
- **The tests cover the core well and the edges poorly.** Authentication, models and schemas are near fully exercised, while user management and the system routes are mostly not. A change to role changes or account deletion would not be caught.
- **The most damaging interface defect is small and invisible to code review of a single file:** the reload comes from the response handler in one file undoing the error message in another. It was found only by running the application. It means a user who mistypes a password gets no feedback at all.
- **The analytics are correct on the cases tested but easy to misread.** Recovery time, status counts and confidence normalisation match expectation. The weaknesses are about presentation (skipped empty periods, non-proportional bars), heuristics (the fraction rule) and definitions that are not stated on screen.
- **Documentation trails the code.** The README describes a configuration option that no longer does anything and omits the steps a newcomer needs first.

6.7 Objective-by-Objective Evaluation

Objective	Evidence	Verdict
U1 Role-based portals with guards	Fourteen guarded routes; build passes; guards read from code	Achieved ; guards not exercised in the browser in this session
U2 Reportee flows	Pages exist; screenshots; API tests cover registration, login and reports	Largely achieved ; failed-login feedback broken (U-1)
U3 Police flows	Pages exist; API tests cover activate and found	Largely achieved ; loading errors unhandled (U-4)
U4 Admin flows	Pages exist; user-management routes only 27% covered	Achieved in implementation ; weakly tested
U5 Correct analytics	Fixed confidence scale; probes on counts and times	Largely achieved ; presentation and definition issues (U-6 to U-10)
U6 Automated tests and coverage	39 tests pass; backend coverage 63.6%	Achieved for the backend; not achieved for the front end
U7 Documentation and run script	README and script exist	Partly achieved : errors and omissions (D-1 to D-4)
U8 Honest evaluation	One browser check, probes, coverage, findings	Achieved for this scope

6.8 Individual Results

All results here concern the front end, analytics, test evidence and documentation. Results about token handling (backend member), the alert channel and status pages (real-time member), and the detector and camera node (AI/vision member) are in their own reports.

6.9 Chapter Summary

The application builds and all 39 Python tests pass, with 63.6% approximate line coverage of the backend. A real browser check found that a wrong password gives no error message, and other checks found unhandled load failures, non-proportional and gappy charts, and README instructions that no longer match the code. None has been fixed in this project.

Chapter 7: Discussion

7.1 Introduction

This chapter interprets the results of Chapter 6, compares them with related practice, states the contributions, and discusses limitations.

7.2 Interpretation of Findings

Research question 1 (can a single-page application with client-side guards give each role only the interface it needs, while the server stays the authority?). Yes for the structure: fourteen routes, each with an explicit role list that mirrors the server's, and navigation that differs by role. The guards are convenience, not security, and the code does not pretend otherwise; the backend report shows where server-side enforcement itself has gaps.

Research question 2 (are the analytics correct and unambiguous?). Correct on the cases tested, not unambiguous. Counts, recovery time and the normalised confidence average behaved as expected. But several figures depend on choices a reader cannot see: what the recovery rate divides by, whether false alerts are in the confidence average, whether a stored 1.0 is 1% or 100%, and whether an empty month is shown. Two of these (U-6 and U-8) can produce a picture that looks right and is not.

Bugs live between files. The login failure (U-1) is not a mistake in the login page or in the API module taken alone. Each does something reasonable: the page shows the server's message, and the API module logs out on a 401. Together they erase the message. It was found by running the application, not by reading the code. This is the main argument for the missing front-end and browser tests: the interface's faults are interactions, and only interaction tests find them.

Coverage tells where to look, not how good the tests are. A 63.6% approximate line coverage with 100% on models and schemas sounds reassuring, but those files mostly run just by loading the application. The informative numbers are the low ones: user management at 27% and system routes at 32%. Those are also routes where the other members' reports found real weaknesses (stale tokens, an open health endpoint), which is consistent with untested code hiding defects.

Documentation is a second interface. The README's instruction to set `CORS_ORIGINS` is worse than no instruction, because it sends a deployer to a setting that does nothing and gives no sign of failure until the front end is blocked in the browser.

7.3 Comparison with Existing Work

No numeric comparison was made. Against the component-library route in Chapter 2, this project has fewer dependencies and a small bundle, and pays for it in hand-written charts that lack the guarantees a charting library gives (proportional bars, empty periods, axes), and in no accessibility support

beyond what was hand-coded. Against typical practice for web applications, a front end with no automated tests is common in student work and is the largest gap here.

7.4 Contributions of the Project

- **Practical:** three role-based portals in fourteen pages; analytics computed from real records; a single-command start and a written quick-start; a body of 39 passing tests.
- **Technical:** a shared confidence-normalisation expression that fixed a real defect and is regression-tested; a small, dependency-light front end (four runtime packages, 82 kB gzipped JavaScript).
- **Analytical:** a list of front-end, analytics and documentation defects with evidence, including one confirmed in a real browser, and a measured coverage profile of the backend that shows where testing is thin.

7.5 Limitations and Implications

Limitation	Implication
One scripted browser check only	Other pages may have defects not found; U-4 and U-5 are from reading, not running
Approximate coverage from a home-made tracer	Figures may differ slightly from coverage.py; treat as indicative
No front-end tests	Every front-end claim rests on inspection or one session
No user evaluation	Usability judgements are the student's own
Severity ratings are the student's own	Not a formal scale
Findings not fixed	The delivered system still has them
Screenshots are from an earlier session	They show the interface as it was then

7.6 Chapter Summary

The front end and analytics work and are lightweight, and their weaknesses lie in interactions, presentation and documentation, not in the core computations. A running application found what reading did not, which argues for interaction tests and for keeping documentation under test.

Chapter 8: Conclusion and Recommendations

8.1 Introduction

This chapter summarises the project, states the conclusions the evidence supports, reviews the objectives, and gives recommendations and future work.

8.2 Summary of the Project

The group built SVDS so that stolen-vehicle reports filed by the public can be matched against plates seen by a camera. This report covered the front end, analytics, testing and documentation: a React single-page application with three role-based portals (fourteen pages), seven analytics endpoints and the page that displays them, a fix for mixed confidence scales, an automated test base of 39 tests, and a README and run script. It was evaluated by a production build, the test suites, an approximate coverage measurement, a scripted browser check and analytics probes.

8.3 Conclusions

1. A small hand-written React application can give each role its own portal, with guards that mirror the server's roles, using four runtime packages and about 82 kB of gzipped JavaScript.
2. The analytics compute real figures from stored records and are correct on the cases tested, after the confidence-scale defect was fixed and regression-tested.
3. The backend test suite passes and exercises about 64% of the backend's lines, unevenly: authentication and models well, user management and system routes poorly.
4. The interface has real defects: a wrong password gives no error message (confirmed in a browser), several page loads fail silently, some charts mislead, and the login form's labels are not associated with its fields.
5. The documentation contains instructions that no longer match the code, including one that would break a deployment.
6. The system is not yet at the standard where it can be handed to users without a fix pass, and it has no automated interface tests to protect such a pass.

8.4 Achievement of Objectives

Objective	Status
U1 Role-based portals with guards	Achieved; guards not browser-tested
U2 Reportee flows	Largely achieved; failed-login feedback broken
U3 Police flows	Largely achieved; load errors unhandled
U4 Admin flows	Achieved in implementation; weakly tested

Objective	Status
U5 Correct analytics	Largely achieved; presentation and definition issues
U6 Tests and coverage	Achieved for the backend; not for the front end
U7 Documentation and run script	Partly achieved
U8 Honest evaluation	Achieved for this scope

Research question 1: yes. Research question 2: correct on the cases tested, not unambiguous.

8.5 Recommendations

In priority order:

1. **Fix the login feedback:** apply the 401 redirect only when the request was not the login call (or when a token existed), so a wrong password shows its message (U-1, U-2).
2. **Correct the README:** remove or fix the CORS_ORIGINS instruction (and make the backend read it), add first-administrator creation, the run script, the tests and the phone camera setup, and correct the camera example (D-1 to D-3).
3. **Handle failures on every data load** with an error message and a Retry, as the Analytics page already does (U-4).
4. **Fill empty periods and make bars proportional** (start at zero, scale to the maximum), or use a charting library (U-6). Label the definitions of recovery rate and average confidence on screen (U-8, U-10).
5. **Record false-positive flags in a column** on the alert, and count that instead of searching audit text (U-9). Store confidence on one scale and migrate the old rows so the normalisation heuristic can be removed (U-8).
6. **Group detections by camera id** (with the latest location), not by camera and location text (U-7).
7. **Associate every form label with its input** and add labels to the login page (U-3).
8. **Add tests where coverage is lowest:** user management and system routes on the backend, then component tests for the guards and forms and one browser test of the login failure (U-11).
9. **Debounce the registry search** and ignore stale responses (U-5).
10. **Remove the absolute path from run-a11.ps1** (D-4).

8.6 Future Work

- Add continuous testing (run the suites and the build on each change).
- Add browser tests for each role's main task using a tool such as Playwright.
- Add a proper charting library, with accessible chart descriptions.
- Measure real usability with officers and members of the public.
- Add pagination totals and export of analytics for reporting.
- Support English and local languages.

References

1. React. "React Documentation." <https://react.dev>
2. React Router. "React Router Documentation." <https://reactrouter.com>
3. Vite. "Vite Documentation." <https://vitejs.dev>
4. Axios. "Axios Documentation." <https://axios-http.com>
5. MDN Web Docs. "Window.localStorage" and "Web accessibility." Mozilla. <https://developer.mozilla.org>
6. W3C. "Web Content Accessibility Guidelines (WCAG) 2.1." <https://www.w3.org/TR/WCAG21/>
7. OWASP Foundation. "OWASP Top 10" and "Cross Site Scripting Prevention Cheat Sheet." <https://owasp.org>
8. Chrome DevTools Protocol documentation. <https://chromedevtools.github.io/devtools-protocol/>
9. pytest. "pytest Documentation." <https://docs.pytest.org>
10. Coverage.py. "Coverage.py Documentation." <https://coverage.readthedocs.io>
11. SQLAlchemy. "SQLAlchemy Documentation." <https://www.sqlalchemy.org>
12. Sommerville, I. (2011). Software Engineering, 9th ed. Pearson.

[Add any interviews, Zambia Police Service correspondence or supervisor notes you used, and format all entries to the department's required style.]

Appendix A: Individual Contribution Statement (Group Projects)

Purpose of this Statement

This statement identifies the work for which the student was primarily responsible. It is consistent with the main report. Work by other group members is acknowledged and is not claimed.

A.1 Project and Group Information

Item	Details
Project Title	SVDS: A Camera-Based Stolen Vehicle Detection System for the Zambia Police Service (built on the VRRS registry)
Student Name	[STUDENT NAME]
Computer Number	[COMPUTER NUMBER]
Supervisor	[SUPERVISOR NAME]
Group Size	4
Group Members	1. Moomba Nicholas Katapazi, [COMPUTER NUMBER] (AI/vision) 2. [NAME], [COMPUTER NUMBER] (Backend and security) 3. [NAME], [COMPUTER NUMBER] (Real-time and integration) 4. [STUDENT NAME], [COMPUTER NUMBER] (Frontend, analytics, testing and documentation)

A.2 Overall Group Project

The group built **SVDS/VRRS**, a system with which the public files stolen-vehicle reports and police officers are alerted automatically when a camera sees a reported vehicle. It has three independently running processes: a FastAPI and PostgreSQL backend, a React frontend with three role-based portals (Reportee, Police, Admin), and a camera node that reads number plates from a phone's video stream using a custom-trained YOLOv8 detector and OCR. The common problem is that stolen-vehicle recovery depends on officers chancing to recognise a plate. The core flow is: the camera node reads a plate, the backend matches it against active reports, and connected police dashboards receive an alert over WebSocket.

A.3 Individual Role and Responsibilities

The group's work was divided into four roles. The student's role was **Role 4: Frontend, analytics, testing and documentation**.

No.	Area of Responsibility	Description
6	Frontend (three portals)	The React web application with role-based routing: Reportee, Police and Admin portals
7	Analytics, testing and documentation	Real computed statistics and charts; the automated tests as evidence; bug fixes; README and run script

Other roles, for reference: **Role 1** AI/vision; **Role 2** Backend and security; **Role 3** Real-time and integration.

A.4 Detailed Individual Contribution

- **Front end:** built the routing and guards, the API module, the shared layouts, the stylesheet and the fourteen pages.
- **Analytics:** wrote the seven analytics endpoints and the analytics page, and fixed the mixed confidence-scale defect with two regression tests.
- **Testing:** ran and interpreted the 39-test suites, measured backend line coverage, ran a scripted browser check, and ran analytics probes.
- **Documentation:** the README and the start-up script; audited the README against the code.
- **Evaluation:** recorded the findings U-1 to U-11 and D-1 to D-4 with evidence.
- **Report:** this report.

A.5 Individual Deliverables

No.	Deliverable	Student's Contribution
1	React front end (vrrs-frontend/src, 24 files, 1,786 lines, plus styles.css)	Designed and implemented
2	Analytics endpoints and page (routers/analytics.py, Analytics.jsx)	Designed and implemented
3	Confidence-scale fix and its two regression tests	Diagnosed, fixed, tested
4	README and run-all.ps1	Written (start-up script also described in the real-time member's report)
5	Coverage measurement and interface/analytics/documentation evaluation	Carried out and documented

A.6 Contribution to the Main System or Research Output

The student produced the part of the system that people use directly, and the statistics and tests that show whether it works. The server rules, tokens and data model were built by the backend member; the alert channel, camera status and health endpoint by the real-time member; the detection model and

camera node by the AI/vision member. The alert connection and toast live in the front end's files but were written by the real-time member.

A.7 Individual Implementation and Technical Work

Modules implemented by the student: App.jsx, main.jsx, AuthContext.jsx, ProtectedRoute.jsx, PortalLayout.jsx, ReporteeLayout.jsx, policeNav.js, services/api.js (except createAlertSocket), the reportee, police and admin pages, styles.css, and routers/analytics.py.

A.8 Individual Testing and Evaluation Contribution

The student ran the 31-test backend suite and the 8-test node suite, measured coverage, ran the browser check and the analytics probes, and wrote Chapter 6. The two analytics regression tests belong to this component. **The student should state which of the remaining test files they wrote themselves**, because the role split assigned tests to this role but the suites also test other members' code.

A.9 Collaboration with Other Group Members

- The **backend member's** API and token are what every page calls; the login response drives the redirect.
- The **real-time member's** alert connection, toast and status pages are mounted and shown by this front end.
- The **AI/vision member's** node stores confidence as a percentage, which the analytics normalisation assumes.

A.10 Contribution Summary

1. **Work primarily completed by the student:** the front end, the analytics endpoints and page, the confidence fix and its tests, the README, and the evaluation.
2. **Work completed jointly with other members:** the run script (with the real-time member); the analytics numbers' consistency with the camera node's confidence scale (with the AI/vision member).
3. **Work primarily completed by other members and used here:** the server rules and tokens; the alert channel and camera status; the detection model and camera node.

A.11 Estimated Contribution

The percentages below are the student's own estimate and must be completed by the student. They were not derived from any record.

Project Activity	Individual Contribution	Collaborative Contribution
Problem Analysis	[%]	[%]
Literature Review	[%]	[%]
Requirements Analysis	[%]	[%]
System/Research Design	[%]	[%]
Implementation / Development	[%]	[%]
Testing and Evaluation	[%]	[%]
Documentation	[%]	[%]
Other	[%]	[%]

A.12 Statement of Authorship and Contribution

I confirm that the contribution described in this statement accurately represents the work that I personally undertook as part of the group project. I have identified collaborative work and have not knowingly claimed the work of another group member as my own.

Student Name: ____ **Computer Number:** ____ **Signature:** ____ **Date:** ____

A.13 Supervisor Verification

I confirm that, to the best of my knowledge, the contribution described above is consistent with the student's participation in the group project.

Supervisor Name: ____ **Signature:** ____ **Date:** ____

Appendix B: User Manual / Installation Guide

This guide covers the front end and how to use each portal. Backend and camera-node set-up are in the other members' reports.

B.1 Requirements

- Node.js and npm.
- The backend running and reachable (default `http://localhost:8000`).

B.2 Install and run

```
cd vrrs-frontend
npm install
cp .env.example .env
npm run dev
```

The app runs at `http://localhost:5173`. `.env` holds `VITE_API_BASE_URL=http://localhost:8000` and `VITE_WS_URL=ws://localhost:8000`. For a production build:

```
npm run build      # writes static files to dist/
```

Serve `dist/` from any static web host, with the two variables set to the deployed backend before building. Note the deployment caveat in finding D-1: the backend's allowed origins are hard-coded and must be changed in its code.

B.3 Using the application

Everyone - Register (public): first and last name, email, telephone (+260 XX XXXXXXX), national ID (NNNNNN/NN/N) and a password of at least 6 characters. Registration creates a reportee. - **Sign in**: you are taken to the portal for your role. *Note*: if you type a wrong password the page reloads with no message (finding U-1); check your details and try again.

Reportee - Report vehicle: enter the plate and vehicle details, when and where it was last seen, and a description; submit. The report starts as "under review". - *My reports*: see your reports and their status (under review, missing, found). - *Profile*: edit your details and change your password.

Police - Vehicle registry: search by plate or owner, filter by status. **Activate** an under-review report so cameras can match it; **mark found** when the vehicle is recovered. - *Live alerts*: review unread alerts; mark read, mark all read, or flag a false positive. A notification appears on any page when a stolen vehicle is detected. - *Cameras*: live feed and node status. - *Analytics*: totals, average recovery time,

average detection confidence, false positives, reports per month, status breakdown and detections by location.

Administrator - *Overview*, *User management* (invite police, change role, deactivate or reactivate, delete with confirmation), *System health*, *Audit log*.

B.4 Reading the analytics

Figure	Meaning
Recovery rate	Found reports as a percentage of all reports, including ones not yet worked
Avg. recovery time	Mean days between filing and being marked found
Avg. detection confidence	Mean confidence of all alerts, including ones later flagged as false
False positives	Alerts flagged as false, counted from the audit log
Reports per month	Months with no reports are not shown; bar heights are capped, so compare the printed counts, not the bar heights

B.5 Troubleshooting

Symptom	Likely cause	Action
Blank or "Loading" page	The backend is not running or unreachable	Start the backend; reload
Signed straight back out	Token expired (24 hours) or backend restarted with a different key	Sign in again
No alert notifications	The alert connection dropped (see the real-time report)	Reload the page
Browser blocks API calls (CORS)	The front end's address is not in the backend's hard-coded list	Add it in the backend's <code>main.py</code>

B.6 Run the checks

```
cd vrrs-frontend && npm run build          # compile check
cd ../vrrs-backend && python -m pytest tests/ -v
cd ../vrrs-node && python -m pytest tests/ -v
```

Appendix C: Additional System Designs / Diagrams

C.1 Further screens (captured earlier from the running application)

SVDS
My Account

- Home
- Report vehicle
- My reports
- My profile**

Demo Reportee
Public user

Logout

My profile

Keep your contact information current for report updates.

Account details

Edit profile

Full name Demo Reportee	Email address demo.reportee@vrrs-thesis.local
Phone number +260971112244	Account role reportee

Change password

Current password	New password

SVDS

Police Portal

Vehicle registry

Live alerts

Cameras

Analytics

VR

Zambia Police

Police Portal

Logout

Cameras

Live feed and connected camera nodes

Active cameras

0/1

Live

Offline nodes

1

Needs attention

Live camera feed

Camera nodes

Node ID	Location
PHONE-NODE-1	Live phone camera offline

SVDS

Admin Portal

Overview

User management

System health

Audit log

VR

Zambia Police

Admin Portal

Logout

System health

Monitor service status, camera nodes, and platform performance

API response time

0.3ms

Normal

DB connections

1/15

Healthy

Disk usage

68.2%

151.4 GB free

Active sessions

21

Right now

Core services

FastAPI backend

http://127.0.0.1:8000

operational

PostgreSQL database

Database reachable

operational

Camera nodes

PHONE-NODE-1 - Live phone camera

Offline

offline

SVDS — Vehicle Registry and Reporting System

1.0.0 OAS 3.1

/openapi.json

Zambia Police Service Vehicle Reporting API

Authorize



Authentication



POST	/auth/register	Register		▼
POST	/auth/login	Login		▼
POST	/auth/logout	Logout		▼

Reports



POST	/reports/	Create Report		▼
GET	/reports/	Get Reports		▼
GET	/reports/{report_id}	Get Report		▼
PATCH	/reports/{report_id}	Update Report		▼
DELETE	/reports/{report_id}	Delete Report		▼
PATCH	/reports/{report_id}/activate	Activate Report		▼
PATCH	/reports/{report_id}/found	Mark Found		▼

Alerts



POST	/alerts/check-plate	Check Plate		▼
GET	/alerts/unread/count	Unread Count		▼
PATCH	/alerts/read-all	Mark All Read		▼
GET	/alerts/	Get Alerts		▼
GET	/alerts/{alert_id}	Get Alert		▼
PATCH	/alerts/{alert_id}/read	Mark Read		▼
PATCH	/alerts/{alert_id}/false-positive	Flag False Positive		▼

Users



POST	/users/invite	Invite Police Officer		▼
GET	/users/me	Get My Profile		▼
PATCH	/users/me	Update My Profile		▼
PATCH	/users/me/password	Change Password		▼
GET	/users/	Get All Users		▼
GET	/users/{user_id}	Get User		▼
DELETE	/users/{user_id}	Delete User		▼
PATCH	/users/{user_id}/role	Change Role		▼
PATCH	/users/{user_id}/toggle-active	Toggle Active		▼

Analytics

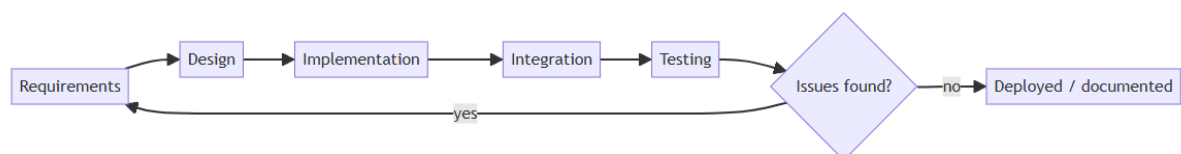


GET	/analytics/summary	Get Summary		▼
GET	/analytics/reports-per-month	Reports Per Month		▼
GET	/analytics/status-breakdown	Status Breakdown		▼

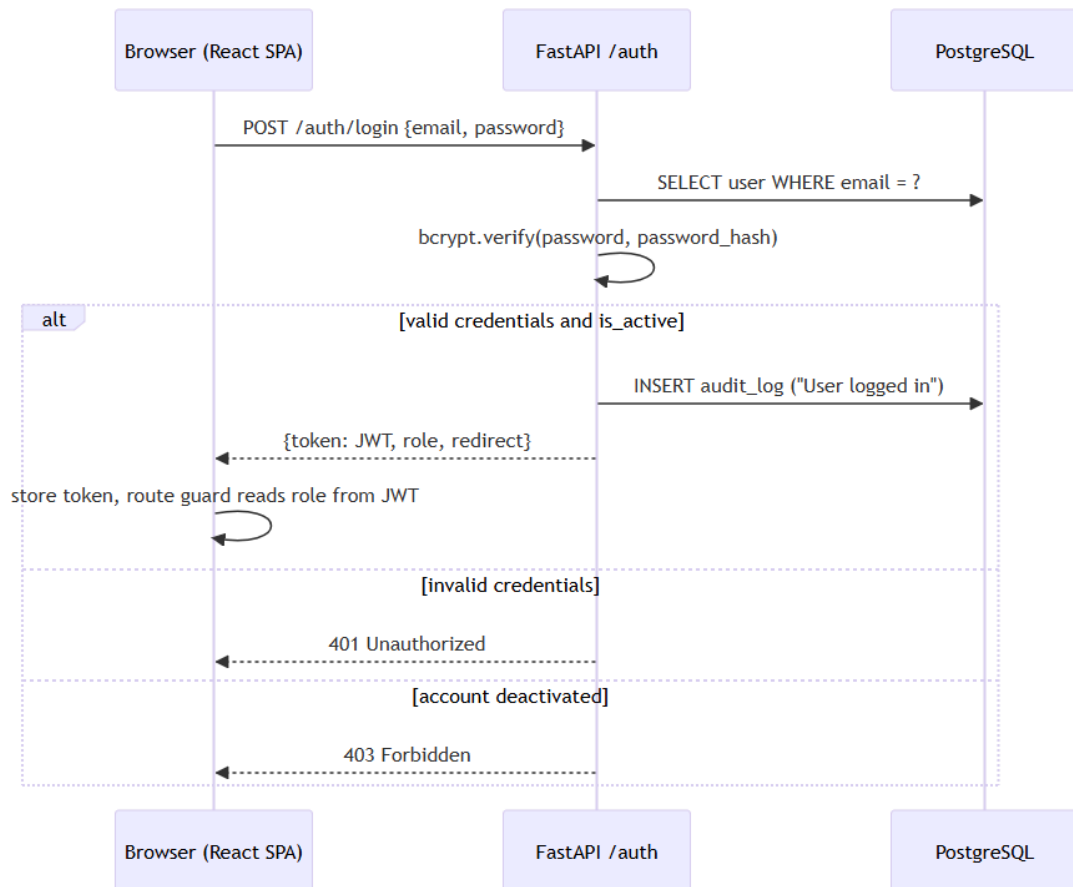
GET	/analytics/detections-per-node	Detections Per Node	🔒	▼
GET	/analytics/alerts-over-time	Alerts Over Time	🔒	▼
GET	/analytics/recovery-time	Avg Recovery Time	🔒	▼
GET	/analytics/user-growth	User Growth	🔒	▼
System				^
GET	/system/audit	Get Audit Log	🔒	▼
GET	/system/camera-nodes	Get Camera Nodes	🔒	▼
GET	/system/live-feed	Get Live Feed	🔒	▼
GET	/system/health	Get System Health	🔒	▼
default				^
GET	/	Root	🔒	▼

Schemas				^
AlertCreate	>	Expand all	object	
AlertOut	>	Expand all	object	
AuditLogOut	>	Expand all	object	
AuthResponse	>	Expand all	object	
HTTPValidationError	>	Expand all	object	
PasswordChange	>	Expand all	object	
ReportCreate	>	Expand all	object	
ReportOut	>	Expand all	object	
ReportUpdate	>	Expand all	object	
RoleChange	>	Expand all	object	
UserInvite	>	Expand all	object	
UserLogin	>	Expand all	object	
UserOut	>	Expand all	object	
UserRegister	>	Expand all	object	
UserUpdate	>	Expand all	object	
ValidationError	>	Expand all	object	

C.2 Iterative development cycle



C.3 Authentication sequence



Appendix D: Additional Test Cases and Results

D.1 Production build output

```
vite v5.4.21 building for production...
107 modules transformed.
dist/index.html                0.43 kB | gzip: 0.30 kB
dist/assets/index-*.css       13.50 kB | gzip: 3.53 kB
dist/assets/index-*.js       263.64 kB | gzip: 82.49 kB
built in 2.31s
```

D.2 Browser check (raw output)

```
marker after wrong-password click -> None    (None means the page reloaded)
visible error text -> 'SVDS - Sign in\nEmail\n\nPassword\n\nSign in\n\nNo account?
Register'
frame navigations after initial load: 2
login response statuses: [200, 401]
labels with for attr: 0 of 2
```

The script started headless Chrome, opened the login page, typed a made-up email and password, clicked Sign in, and read the page. It wrote nothing to the database (the server rejected the login). The backend and dev server were started for the check and stopped afterwards.

D.3 Analytics probes (raw output)

```
PROBE reports-per-month (6): [('May',1), ('Jul',1), ('Aug',1), ('Sep',2)]
PROBE summary reports:
{'total':5,'missing':3,'found':1,'under_review':1,'new_this_week':1,'recovery_rate':20.0}
PROBE recovery-time:
{'avg_days':6.0,'fastest_days':6,'slowest_days':6,'total_recovered':1}
PROBE detections-per-node rows for one camera: [('CAM-X','Cairo Rd',1), ('CAM-X','Cairo
Road',1)]
PROBE confidence stored 1.0 -> avg shown: 100.0
PROBE alerts-over-time (30d) points: 2 for 2 alert-days
PROBE false_positives after unrelated audit row containing the phrase: 1
PROBE months=0 -> 422 | days=3 -> 422
```

D.4 Coverage (raw output)

COV auth.py 24/26 = 92.3%	COV routers\alerts.py 75/110 = 68.2%
COV database.py 12/16 = 75.0%	COV routers\analytics.py 50/72 = 69.4%
COV main.py 31/39 = 79.5%	COV routers\auth.py 42/43 = 97.7%

```
COV middleware.py 23/27 = 85.2% COV routers\model.py      0/38 = 0.0%
COV model.py 0/23 = 0.0%      COV routers\reports.py 65/87 = 74.7%
COV models.py 61/61 = 100.0%  COV routers\system.py  37/117 = 31.6%
COV schemas.py 114/114 = 100.0% COV routers\users.py  31/115 = 27.0%
COV TOTAL 565/888 = 63.6%    (pytest exit 0; 31 passed)
```

Appendix E: Data Collection Instruments

Not applicable. No questionnaires, interviews or surveys were used.

Appendix F: Additional Code / Configuration

F.1 The response handler behind finding U-1 (services/api.js)

```
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.clear();
      window.location.href = "/login";
    }
    return Promise.reject(error);
  }
);
```

A wrong password also returns 401, so the page is reloaded before the login form can display the server's message. A fix is to skip the redirect when the request was to /auth/login.

F.2 Bar height (pages/police/Analytics.jsx)

```
style={{ height: `${Math.max(20, Math.min((m.count || 1) * 10, 170))}px` }}
```

Counts of 1 and 2 both give 20 px; counts of 17 or more all give 170 px (finding U-6).

F.3 Confidence normalisation (routers/analytics.py)

```
NORMALIZED_CONFIDENCE = case(
  (Alert.confidence_score <= 1, Alert.confidence_score * 100),
  else_=Alert.confidence_score,
)
```

F.4 Front-end environment template (vrss-frontend/.env.example)

```
VITE_API_BASE_URL=http://localhost:8000
VITE_WS_URL=ws://localhost:8000
```

Appendix G: Other Supporting Material

G.1 Group role split

Role	Work areas	Focus
1. AI/vision	Detection model; camera node	YOLOv8n detector, OCR pipeline
2. Backend and security	Backend API and database; security and access control	FastAPI, PostgreSQL, JWT, roles
3. Real-time and integration	Real-time alerts and monitoring; phone-to-server wiring	WebSocket, camera status, feed proxy, health
4. Frontend	Frontend portals; analytics, testing and documentation	This report

G.2 Findings summary

ID	Finding	Severity
U-1	Failed login shows no error (page reloads)	High
U-2	Every 401 treated as an expired session (cause of U-1)	High
U-3	Login labels not associated with inputs	Medium
U-4	Several data loads have no error handling	Medium
U-5	Registry search on every keystroke, no stale-reply guard	Low
U-6	Empty periods omitted; bars not proportional	Medium
U-7	One camera split into several rows	Low
U-8	Stored 1.0 shown as 100%	Low
U-9	False positives counted by searching audit text	Low
U-10	Recovery rate and confidence definitions not stated on screen	Low
U-11	No automated front-end tests	Medium
D-1	README CORS_ORIGINS instruction has no effect	High (deployment)
D-2	README omits run script, first admin, tests, phone app	Medium
D-3	README camera example incomplete	Low
D-4	Absolute developer path in run-all.ps1	Medium